zeros( ) Function

zeros ( ) Function is used to create an array with all zeros.

Syntax:-

numpy.zeros(shape, dtype=float, order='C')

Where,

shape – shape of new array. It can be an int which will represent number of elements or can be tuple of int. ex:- 5, (5,) (3, 1)

dtype – The desired data-type for the array.

Order - Whether to store multi-dimensional data in row-major (C-style) or column-major (Fortran-style) order in memory. It can be C or F.

Creating Array using zeros ( ) Function

Syntax:-

from numpy import *

array_name = zeros(shape, dtype=float)

Ex:-

from numpy import *

a = zeros(5)

a = zeros(5, dtype=int)

a = zeros((3, 2))

0.0	0.0	0.0	0.0	0.0
a[0]	a[1]	a[2]	a[3]	a[4]

Accessing One-D Array Elements

*from numpy import **

a = zeros(5)

print(*a*[0])

print(*a*[1])

print(*a*[2])

print(*a*[3])

print(*a*[4])

0.0	0.0	0.0	0.0	0.0
a[0]	a[1]	a[2]	a[3]	a[4]

<u>Accessing using for loop</u>

*from numpy import **

a = zeros(5)

<u>Without index</u>

```
for el in a:
        print(el)
```

<u>With index</u>

```
n = len(a)
for i in range(n):
        print(a[i])
```

Accessing using while loop

```
from numpy import *
a = zeros(5)

n = len(a)
i = 0
while i < n :
        print(a[i])
        i+=1
```

ones( ) Function

ones ( ) Function is used to create an array with all 1s.

Syntax:-

numpy.ones(shape, dtype=float, order='C')

Where,

shape – shape of new array. It can be an int which will represent number of elements or can be tuple of int.

dtype – The desired data-type for the array. Default is float.

Order - Whether to store multi-dimensional data in row-major (C-style) or column-major (Fortran-style) order in memory. It can be C or F.

Creating Array using ones ( ) Function

Syntax:-

from numpy import *

array_name = ones(shape, dtype=float)

Ex:-

from numpy import *

a = ones(5)

a = ones(5, dtype=int)

a = ones((3, 2))

1.	1.	1.	1.	1.
a[0]	a[1]	a[2]	a[3]	a[4]

Accessing One-D Array Elements

```
from numpy import *
a = ones(5)
print(a[0])
print(a[1])
print(a[2])
print(a[3])
print(a[4])
```

1.	1.	1.	1.	1.
a[0]	a[1]	a[2]	a[3]	a[4]

Accessing using for loop

*from numpy import **

a = ones(5)

Without index

```
for el in a:
        print(el)
```

With index

```
n = len(a)
for i in range(n):
        print(a[i])
```

Accessing using while loop

```
from numpy import *
a = ones(5)

n = len(a)
i = 0
while i < n :
		print(a[i])
		i+=1
```